CHEAT SHEET · JS INTERVIEW #09 @specsycoder

var vs let in a loop

Why it prints 3 3 3 instead of 0 1 2

var-loop.js

```
1 for (var i = 0; i < 3; i++) {
2   setTimeout(() => console.log(i), 0);
3 }
```

output 3 3 3

let-loop.js

```
1 for (let i = 0; i < 3; i++) {
2   setTimeout(() => console.log(i), 0);
3 }
```

output 0 1 2

Why it happens

01 var = ONE i

var is function scoped, so the whole loop shares a single i.

02 Loop finishes first

Timers run later. By then the loop is done and i is 3.

03 Closure

Each callback remembers the variable i, not its value at that moment.

04 let = new i each round

let is block scoped, so every iteration gets its own i.

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

Closures, simply

A function remembers the variables around it

old-fixes.js

```
// before let existed: IIFE
for (var i = 0; i < 3; i++) {
  ((j) => setTimeout(() => console.log(j)))(i);
}
// or pass i as a timer argument
for (var i = 0; i < 3; i++) {
  setTimeout((j) => console.log(j), 0, i);
}
```

variation	prints	why
var + setTimeout	3 3 3	one shared i
let + setTimeout	0 1 2	new i per iteration
var + IIFE (j)	0 1 2	j is a copy per call
const i in for (;;)	TypeError	i++ reassigns a const

Traps to mention

- **It is not about the 0ms delay.** Even with 1000ms, var still prints 3 3 3.
- **Closures capture variables, not values.** They read the latest value when they run.
- **Same bug with click handlers** attached inside a var loop.

Your 20-second answer

"var is function scoped, so the loop has only one i. The timers run after the loop ends, and their closures all read that same i, which is 3. let is block scoped, so each iteration gets a new i, and it prints 0, 1, 2."